

AWS / クラウドネイティブ受託開発

# コンテナ Web アーキテクチャ 構築ケイパビリティのご紹介

CloudFront + ALB + ECS Fargate + Aurora Serverless v2

リファレンスアーキテクチャに基づく、設計から構築・運用までの技術力のご説明

---

御社名（要編集）

2026 年 6 月 初版ドラフト

# 私たちが提供できる価値

クラウドネイティブな Web システムを、要件定義・設計から構築・運用改善まで一気通貫でご提供します。

01

## クラウド設計力

AWS ベストプラクティスに沿った、拡張性・可用性の高いアーキテクチャ設計。

02

## コンテナ構築力

ECS Fargate による、サーバー管理不要でスケーラブルな実行基盤の構築。

03

## IaC 自動化

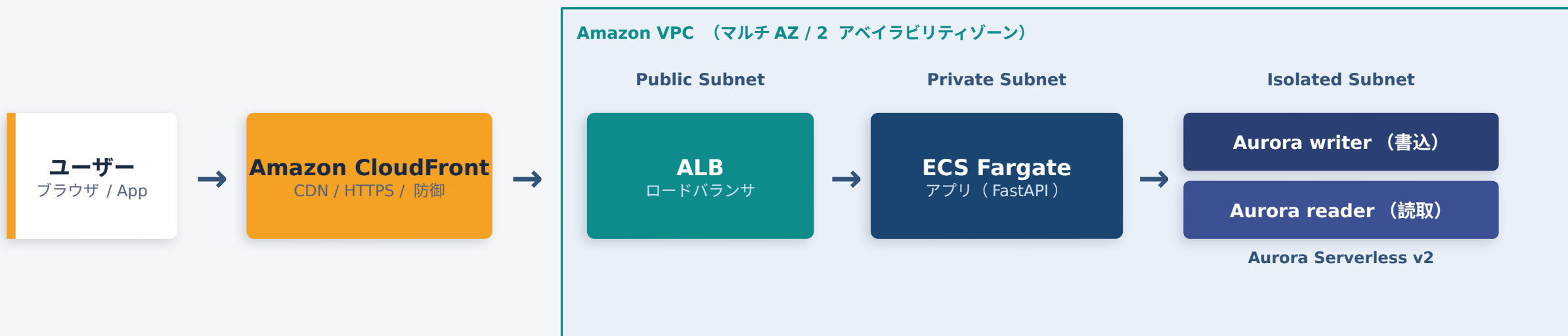
AWS CDK（TypeScript）でインフラをコード化し、再現性と変更管理を実現。

04

## 堅牢 & 低コスト

最小権限・サーバーレス v2 で、セキュアかつ無駄のないコスト構造に。

# リファレンスアーキテクチャ：コンテナ Web 構成



**設計のポイント：** 通信は常に HTTPS の CloudFront 経由。ALB は公開しつつも CloudFront からのみ受信を許可。DB は出口のない分離サブネットに配置し、読み書きを分離して負荷を分散。

# 各コンポーネントの役割と採用理由

| コンポーネント                   | 役割                         | 採用理由（お客様メリット）             |
|---------------------------|----------------------------|---------------------------|
| Amazon CloudFront         | 世界中のエッジから配信・HTTPS 終端・キャッシュ | 高速化・HTTPS・DDoS 防御を低コストで実現 |
| Application Load Balancer | L7 ロードバランサ。リクエストを振り分け      | 可用性向上・複数サービスの集約・ヘルスチェック   |
| Amazon ECS Fargate        | サーバー管理不要のコンテナ実行基盤          | 運用負荷を抑えつつ自動スケール・無停止更新     |
| Aurora Serverless v2      | 自動伸縮するリレーショナル DB（読み書き分離）   | 負荷に応じて自動拡張、アイドル時は停止し低コスト  |

※ いずれも AWS マネージドサービスを採用し、構築・運用コストと障害リスクを最小化しています。

# 対応可能な案件タイプ

本構成を土台に、以下のような幅広い Web ・ 業務システム案件に対応できます。



## SaaS ・ Web サービス基盤

マルチテナント型の製品バックエンド構築・運用。



## EC サイト・予約システム

商品 / 注文 / 在庫など、読み書きの多い業務に対応。



## API バックエンド

モバイル / SPA 向けの REST ・ GraphQL API 基盤。



## クラウド移行・刷新

既存システムのコンテナ化と AWS への移行。



## 社内業務システム

管理画面・ワークフロー等の社内基幹システム。



## データ処理・連携基盤

バッチ処理やデータ連携、外部 API 連携など。

# 技術領域と対応範囲

## AWS

- **ネットワーク設計**

VPC ・ サブネット分離 ・ セキュリティグループ ・ ルーティング

- **コンテナ**

ECS / Fargate / ECR によるコンテナ実行基盤

- **データベース**

Aurora ・ RDS ・ DynamoDB の設計と読み書き分離

- **サーバーレス**

Lambda ・ API Gateway によるイベント駆動構成

- **IaC ・ 自動化**

AWS CDK ( TypeScript ) ／ CloudFormation

- **セキュリティ／監視**

IAM 最小権限 ・ Secrets Manager ・ WAF ・ CloudWatch

## Python

- **Web / API 開発**

FastAPI ・ Flask による API ・ Web アプリ実装

- **DB 連携**

psycopg2 ・ SQLAlchemy 等での RDB 連携

- **サーバーレス関数**

Lambda 上での業務ロジック ・ 連携処理

- **データ処理 ・ 自動化**

バッチ ・ ETL ・ 運用自動化スクリプト

- **認証 ・ 認可**

Cognito ・ JWT 等による認証 / 権限制御の実装

- **品質**

型ヒント ・ テスト ・ コンテナ化を前提とした実装

実務経験 約 10 年 ／ AWS 認定ソリューションアーキテクト アソシエイト ( SAA ) 保有 ／ 設計 ・ 実装 ・ レビューまで一貫対応

## お客様の課題と、私たちの解決アプローチ

### 課題

アクセス急増に耐えられない



### 解決

Fargate と Aurora Serverless v2 の自動スケールで需要に追従

### 課題

インフラコストが高い・読めない



### 解決

サーバーレス v2 + 従量課金、アイドル時は自動停止して無駄を削減

### 課題

セキュリティに不安がある



### 解決

ネットワーク分離・最小権限 IAM ・ Secrets Manager ・ オリジン保護

### 課題

構築 / 運用が属人化している



### 解決

AWS CDK でコード化し、誰でも再現・変更影響を事前確認

### 課題

リリースで停止・失敗が怖い



### 解決

ローリングデプロイで無停止更新、ヘルスチェックで自動復旧

# 非機要件への作り込み

「動く」だけでなく、本番運用に求められる品質を標準で作り込みます。

## セキュリティ

- 3層ネットワーク分離（公開 / 非公開 / 分離）
- ALB は CloudFront 経由のみ許可
- 認証情報は Secrets Manager 管理
- IAM は最小権限で付与

## コスト最適化

- Serverless v2 Min0 で自動停止
- NAT/ALB 等の固定費を可視化
- タグでコストを按分・分析
- 従量課金で無駄を排除

## 可用性

- マルチ AZ で単一障害点を回避
- ALB ヘルスチェックで異常隔離
- リーダー昇格で自動切替
- 無停止のローリング更新

## 運用性

- AWS CDK でインフラをコード化
- cdk diff で変更影響を事前確認
- CloudWatch ヘログ集約
- 環境の再現・複製が容易

本番運用を見据えた標準化により、リリース後の安定稼働と継続的な改善を支えます。



# コンテナとサーバーレス、両構成に対応

案件の特性に応じて最適なアーキテクチャを選定・ご提案します。両構成の知見を有しています。

## CONTAINER

### コンテナ構成

- CloudFront + ALB + ECS Fargate + Aurora
- 常時稼働・長時間処理・WebSocket
- リレーショナルデータ・複雑なクエリ

向き： 安定した負荷の Web/ 業務システム

## SERVERLESS

### サーバーレス構成

- API Gateway + Lambda + DynamoDB
- イベント駆動・スパイク的なアクセス
- 運用レス・極小コストで小さく始める

向き： 変動の大きい API ・軽量サービス

**選定の考え方：** 処理時間・常時接続の有無・データモデル（RDB/NoSQL）・トラフィック特性から、最適解をご提案します。

## SUMMARY

# まとめ / 次のステップ

1

### 設計から運用まで一気通貫

AWS のベストプラクティスに沿い、クラウドネイティブな Web システムを構築します。

2

### コンテナ × サーバーレス両対応

案件特性に応じて最適なアーキテクチャを選定・ご提案します。

3

### 品質を標準装備

セキュリティ・コスト最適化・可用性・IaC による運用性を最初から作り込みます。

## 次のステップ

受託開発のご相談・既存システムの課題整理など、まずはお打ち合わせにて貴社のご要望をお聞かせください。